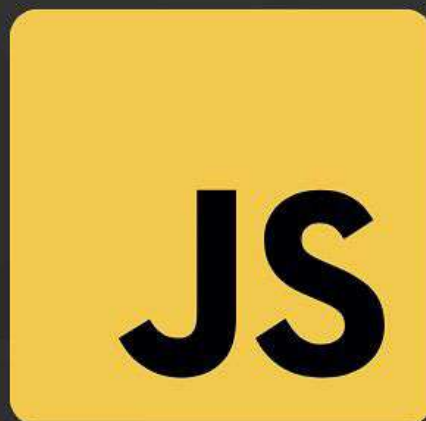




Miss Techie
@misstechie15

Shallow Copy & Deep Copy



Swipe →

Let's talk about **shallow copy** and **deep copy** in a simple way, using a real-life example.

Imagine you have a **drawing** of a house :
The house has a **main building** and a **garden** with some flowers.

You want to make a **copy** of this drawing.

In **shallow copy** is like tracing the outline of the house and garden but **not** the details inside.

In **deep copy** is like making a **completely** new drawing of the house and garden, including all the details.



Miss Techie
@misstechie15

Next →

Shallow Copy

A **shallow copy** creates a new object, but it **does not** create copies of the objects inside the original object.

Instead, it **copies** the **references** (or memory addresses) of the nested objects from the original object to the new object.

This means that nested objects or arrays are **not completely cloned**, and any changes to the nested elements will be **reflected** in both the original and the copied object.



Ways to Create a Shallow Copy

Here are several methods you can use to perform a shallow copy:

1. Using `Object.assign()`

It copies all enumerable properties from the source object to the target object.

```
let shallowCopy= Object.assign({}, originalObject);
```

2. Using the Spread Operator (...)

The spread operator (...) is a more modern and concise way to create a shallow copy of an object or an array.

```
let shallowCopy= { ...originalObject };
```



Shallow Copy in Real-life Scenario

1. You have a **photo** of your family on your phone.
2. You send a **link** to the photo to your friend (**this is like a shallow copy**).
3. If you **edit** the photo (like adding a filter), your friend will see the **edited photo** because they're looking at the same original photo through the **link**.



```
1 let originalObject = { name: "Alice", age: 30 };
2
3 // Shallow copy using Object.assign()
4 let shallowCopy = Object.assign({}, originalObject);
5
6 console.log(shallowCopy); // { name: "Alice", age: 30 }
```



Deep Copy

A **deep copy** creates a **completely** new object and recursively copies all objects found within the original object.

This means that it creates **independent copies** of all nested objects, so changes to the **original object** or its nested objects **do not affect** the deep copy, and vice versa.

Deep copy ensures complete **independence** between the original and the copied object.



Ways to Create a Deep Copy

Here are several methods you can use to perform a deep copy:

1. Using `JSON.parse()` and `JSON.stringify()`

This is one of the simplest ways to perform a deep copy when working with JSON-like data.

```
let deepCopy =  
JSON.parse(JSON.stringify(originalObject));
```

2. Using `Lodash's _.cloneDeep()`

The Lodash library provides a built-in function called `_.cloneDeep()` for deep cloning.

```
let deepCopyObj = _.cloneDeep(originalObject);
```



Deep Copy in Real-life Scenario

1. You have a **photo** of your family on your phone.
2. You print out the photo and give the **printed copy** to your friend (this is like a deep copy).
3. If you edit the photo on your phone (like adding a filter), the printed copy **won't change** because it's a separate, physical copy.



```
1  let originalObject = { name: "Alice", age: 30 };
2
3  // Deep copy using JSON.parse() and JSON.stringify()
4  let deepCopy = JSON.parse(JSON.stringify(originalObject));
5
6  console.log(deepCopy); // { name: "Alice", age: 30 }
```



Miss Techie
@misstechie15

Next →